



SIMATS ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES



SMART HOSPITAL BED & PATIENT MONITORING MANAGEMENT SYSTEM

DSA01 – Object Oriented Programming with C++

Department of Computer Science and Engineering

ASSIGNMENT REPORT

Submitted by: DHARSHIDHA S

Register Number: 1911260095

Submitted to: Dr.PARIMALA KANAGA DEVAN K

1. Problem Understanding

The assignment requires a menu-driven, object-oriented C++ application for a medium-sized hospital. The system registers patients, manages a fixed set of beds, allocates suitable beds, creates admission records, records monitoring/service usage, calculates bills, discharges patients, compares admissions, and generates a bed-utilization report.

- Patient categories: General Patient, ICU Patient, and Emergency Patient.
- The Patient class is the base class; derived classes override patient-specific behaviour.
- HospitalBed stores bed number, ward type, and occupancy status.
- Admission connects a patient with a bed and stores stay/service information.
- Hospital manages collections, allocation, reports, and the menu.
- The implementation demonstrates encapsulation, inheritance, polymorphism, constructors, destructor, static members, collections, operator overloading, validation, and control structures.

2. Deliverable 1 – Pseudocode

START

Create Hospital object with fixed number of beds

REPEAT

Display menu

Read user choice

IF choice = 1

Read patient details

Ask for patient category

Create GeneralPatient / ICUPatient / EmergencyPatient

Store patient in hospital collection

Increase registered patient count

ELSE IF choice = 2

Display all registered patients

ELSE IF choice = 3

Display all beds and availability

ELSE IF choice = 4

Read patient ID

Check whether patient exists and is not already admitted

Find a suitable free bed according to category/priority

If bed is available

Mark bed occupied

Create admission record

Else

Display unavailable-bed message

ELSE IF choice = 5

Read patient ID

Find active admission

Read monitoring/service units

Update admission usage

ELSE IF choice = 6

Read patient ID

Find active admission

Calculate and display current bill

ELSE IF choice = 7

Read patient ID

Find active admission

Calculate final bill

Close admission

Release bed

ELSE IF choice = 8

Display admission records

ELSE IF choice = 9

Read two admission IDs

Compare records using overloaded operator >

ELSE IF choice = 10

Calculate total, occupied and available beds

Calculate utilization percentage

Display active admissions and revenue

ELSE IF choice = 0

EXIT

ELSE

Display invalid choice

UNTIL choice = 0

END

3. Deliverable 2 – Complete C++ Implementation

The following implementation uses the required structure-based organization: related data and functions are grouped into structures, while inheritance and virtual functions are used to demonstrate runtime polymorphism. It is designed as a single source file and includes validation and the requested menu operations.

Source Code:

```
#include <iostream>
#include <vector>
#include <memory>
#include <iomanip>
#include <string>
#include <algorithm>
using namespace std;

// =====
// PATIENT HIERARCHY
// Structure method: data + related functions are grouped
// inside structures. Virtual functions provide polymorphism.
// =====

struct Patient {
protected:
    int id;
    string name;
    int age;
    string diagnosis;

public:
    static int totalRegistered;

    Patient() : id(0), name("Unknown"), age(0), diagnosis("Not specified") {
        ++totalRegistered;
    }

    Patient(int id, const string& name, int age, const string& diagnosis)
        : id(id), name(name), age(age), diagnosis(diagnosis) {
        ++totalRegistered;
    }

    Patient(const Patient& other)
        : id(other.id), name(other.name), age(other.age),
          diagnosis(other.diagnosis) {
        ++totalRegistered;
    }

    virtual ~Patient() {
        // Patient cleanup can be handled here if required.
    }

    int getId() const { return id; }
    string getName() const { return name; }

    virtual string getCategory() const = 0;
    virtual double getDailyCharge() const = 0;
    virtual double getServiceCharge() const = 0;
    virtual int getPriority() const = 0;
```

```

    virtual int requiredMonitoring() const = 0;

    virtual void display() const {
        cout << left << setw(8) << id
            << setw(20) << name
            << setw(6) << age
            << setw(16) << getCategory()
            << setw(18) << diagnosis
            << '\n';
    }
};

int Patient::totalRegistered = 0;

struct GeneralPatient : public Patient {
    GeneralPatient() : Patient() {}
    GeneralPatient(int id, const string& name, int age, const string& diagnosis)
        : Patient(id, name, age, diagnosis) {}
    GeneralPatient(const GeneralPatient& other) : Patient(other) {}

    string getCategory() const override { return "General"; }
    double getDailyCharge() const override { return 2000.0; }
    double getServiceCharge() const override { return 300.0; }
    int getPriority() const override { return 1; }
    int requiredMonitoring() const override { return 1; }
};

struct ICUPatient : public Patient {
    ICUPatient() : Patient() {}
    ICUPatient(int id, const string& name, int age, const string& diagnosis)
        : Patient(id, name, age, diagnosis) {}
    ICUPatient(const ICUPatient& other) : Patient(other) {}

    string getCategory() const override { return "ICU"; }
    double getDailyCharge() const override { return 7500.0; }
    double getServiceCharge() const override { return 1000.0; }
    int getPriority() const override { return 3; }
    int requiredMonitoring() const override { return 3; }
};

struct EmergencyPatient : public Patient {
    EmergencyPatient() : Patient() {}
    EmergencyPatient(int id, const string& name, int age, const string& diagnosis)
        : Patient(id, name, age, diagnosis) {}
    EmergencyPatient(const EmergencyPatient& other) : Patient(other) {}

    string getCategory() const override { return "Emergency"; }
    double getDailyCharge() const override { return 5000.0; }
    double getServiceCharge() const override { return 750.0; }
    int getPriority() const override { return 4; }
    int requiredMonitoring() const override { return 2; }
};

// =====
// BED
// =====

struct HospitalBed {
    int bedNo;
    string wardType;
    bool occupied;
    int patientId;
};

```

```

HospitalBed() : bedNo(0), wardType("General"), occupied(false), patientId(-1) {}

HospitalBed(int no, const string& ward)
    : bedNo(no), wardType(ward), occupied(false), patientId(-1) {}

bool canAccept(const Patient& p) const {
    if (occupied) return false;
    if (p.getCategory() == "ICU") return wardType == "ICU";
    if (p.getCategory() == "Emergency")
        return wardType == "Emergency" || wardType == "General";
    return wardType == "General";
}

void occupy(int pid) {
    occupied = true;
    patientId = pid;
}

void release() {
    occupied = false;
    patientId = -1;
}

void display() const {
    cout << left << setw(8) << bedNo
        << setw(14) << wardType
        << setw(12) << (occupied ? "Occupied" : "Available")
        << setw(12) << (occupied ? to_string(patientId) : "-")
        << '\n';
}
};

// =====
// ADMISSION
// =====

struct Admission {
    int admissionId;
    int patientId;
    int bedNo;
    int days;
    int serviceUnits;
    bool active;
    double finalBill;

    Admission()
        : admissionId(0), patientId(-1), bedNo(-1), days(0),
          serviceUnits(0), active(false), finalBill(0) {}

    Admission(int aid, int pid, int bno)
        : admissionId(aid), patientId(pid), bedNo(bno), days(1),
          serviceUnits(0), active(true), finalBill(0) {}

    Admission(const Admission& other)
        : admissionId(other.admissionId), patientId(other.patientId),
          bedNo(other.bedNo), days(other.days),
          serviceUnits(other.serviceUnits), active(other.active),
          finalBill(other.finalBill) {}

    ~Admission() {
        // Admission object cleanup point.
    }
}

```

```

void updateUsage(int extraUnits) {
    if (extraUnits > 0) serviceUnits += extraUnits;
}

double calculateBill(const Patient& p) const {
    return days * p.getDailyCharge()
        + serviceUnits * p.getServiceCharge();
}

void close(const Patient& p) {
    finalBill = calculateBill(p);
    active = false;
}

// Meaningful operator overload: compare admissions by bill.
bool operator>(const Admission& other) const {
    return finalBill > other.finalBill;
}

// Meaningful operator overload: formatted admission report.
friend ostream& operator<<(ostream& os, const Admission& a) {
    os << left << setw(8) << a.admissionId
        << setw(10) << a.patientId
        << setw(8) << a.bedNo
        << setw(8) << a.days
        << setw(10) << a.serviceUnits
        << setw(12) << (a.active ? "Active" : "Closed")
        << setw(12) << fixed << setprecision(2) << a.finalBill;
    return os;
}
};

// =====
// HOSPITAL
// =====

struct Hospital {
    vector<unique_ptr<Patient>> patients;
    vector<HospitalBed> beds;
    vector<Admission> admissions;
    static int totalAdmissions;

    Hospital() {
        // Fixed bed collection for a medium-sized prototype.
        beds.emplace_back(101, "General");
        beds.emplace_back(102, "General");
        beds.emplace_back(103, "General");
        beds.emplace_back(201, "ICU");
        beds.emplace_back(202, "ICU");
        beds.emplace_back(301, "Emergency");
    }

    ~Hospital() {
        // Active beds are released before Hospital is destroyed.
        for (auto& bed : beds)
            if (bed.occupied) bed.release();
    }

    Patient* findPatient(int id) {
        for (auto& p : patients)
            if (p->getId() == id) return p.get();
        return nullptr;
    }
}

```

```

Admission* findAdmission(int patientId) {
    for (auto& a : admissions)
        if (a.patientId == patientId && a.active) return &a;
    return nullptr;
}

HospitalBed* findBed(int bedNo) {
    for (auto& b : beds)
        if (b.bedNo == bedNo) return &b;
    return nullptr;
}

bool alreadyRegistered(int id) {
    return findPatient(id) != nullptr;
}

void registerPatient() {
    int id, age, type;
    string name, diagnosis;

    cout << "Enter Patient ID: ";
    cin >> id;
    if (alreadyRegistered(id)) {
        cout << "Patient ID already exists.\n";
        return;
    }

    cout << "Enter Name: ";
    cin.ignore();
    getline(cin, name);

    cout << "Enter Age: ";
    cin >> age;
    if (age < 0 || age > 120) {
        cout << "Invalid age.\n";
        return;
    }

    cout << "Enter Diagnosis: ";
    cin.ignore();
    getline(cin, diagnosis);

    cout << "1. General Patient\n";
    cout << "2. ICU Patient\n";
    cout << "3. Emergency Patient\n";
    cout << "Select category: ";
    cin >> type;

    if (type == 1)
        patients.push_back(make_unique<GeneralPatient>(id, name, age, diagnosis));
    else if (type == 2)
        patients.push_back(make_unique<ICUPatient>(id, name, age, diagnosis));
    else if (type == 3)
        patients.push_back(make_unique<EmergencyPatient>(id, name, age, diagnosis));
    else {
        cout << "Invalid category.\n";
        return;
    }

    cout << "Patient registered successfully.\n";
}

void displayPatients() const {

```



```

    if (patients.empty()) {
        cout << "No patients registered.\n";
        return;
    }

    cout << left << setw(8) << "ID"
        << setw(20) << "Name"
        << setw(6) << "Age"
        << setw(16) << "Category"
        << setw(18) << "Diagnosis" << '\n';
    cout << string(68, '-') << '\n';

    for (const auto& p : patients) p->display();

    cout << "\nTotal registered patients: "
        << Patient::totalRegistered << '\n';
}

void displayBeds() const {
    cout << left << setw(8) << "Bed"
        << setw(14) << "Ward"
        << setw(12) << "Status"
        << setw(12) << "Patient ID" << '\n';
    cout << string(46, '-') << '\n';

    for (const auto& b : beds) b.display();
}

HospitalBed* allocateBed(Patient& p) {
    HospitalBed* best = nullptr;

    // First try exact ward match.
    for (auto& b : beds) {
        if (b.canAccept(p)) {
            if (best == nullptr || b.wardType == p.getCategory())
                best = &b;
        }
    }

    if (best) best->occupy(p.getId());
    return best;
}

void admitPatient() {
    int pid;
    cout << "Enter Patient ID: ";
    cin >> pid;

    Patient* p = findPatient(pid);
    if (!p) {
        cout << "Patient not found.\n";
        return;
    }

    if (findAdmission(pid)) {
        cout << "Patient is already admitted.\n";
        return;
    }

    HospitalBed* bed = allocateBed(*p);
    if (!bed) {
        cout << "No suitable free bed is available.\n";
        return;
    }
}

```

```

    }

    int aid = static_cast<int>(admissions.size()) + 1;
    admissions.emplace_back(aid, pid, bed->bedNo);
    ++totalAdmissions;

    cout << "Admission successful. Bed allocated: "
          << bed->bedNo << '\n';
}

void updateMonitoring() {
    int pid, units;
    cout << "Enter Patient ID: ";
    cin >> pid;

    Admission* a = findAdmission(pid);
    Patient* p = findPatient(pid);

    if (!a || !p) {
        cout << "Active admission not found.\n";
        return;
    }

    cout << "Patient type: " << p->getCategory() << '\n';
    cout << "Suggested monitoring units/checks: "
          << p->requiredMonitoring() << " per day\n";

    cout << "Enter additional service/monitoring units: ";
    cin >> units;

    if (units <= 0) {
        cout << "Units must be positive.\n";
        return;
    }

    a->updateUsage(units);
    cout << "Monitoring/service usage updated.\n";
}

void calculateCurrentBill() {
    int pid;
    cout << "Enter Patient ID: ";
    cin >> pid;

    Admission* a = findAdmission(pid);
    Patient* p = findPatient(pid);

    if (!a || !p) {
        cout << "Active admission not found.\n";
        return;
    }

    cout << fixed << setprecision(2);
    cout << "Current Bill = Rs. " << a->calculateBill(*p) << '\n';
}

void dischargePatient() {
    int pid;
    cout << "Enter Patient ID: ";
    cin >> pid;
}

```

```

Admission* a = findAdmission(pid);
Patient* p = findPatient(pid);

if (!a || !p) {
    cout << "Active admission not found.\n";
    return;
}

int days;
cout << "Enter total stay in days: ";
cin >> days;

if (days <= 0) {
    cout << "Days must be positive.\n";
    return;
}

a->days = days;
a->close(*p);

HospitalBed* bed = findBed(a->bedNo);
if (bed) bed->release();

cout << fixed << setprecision(2);
cout << "Patient discharged successfully.\n";
cout << "Final Bill = Rs. " << a->finalBill << '\n';
cout << "Bed " << a->bedNo << " released.\n";
}

void displayAdmissions() const {
    if (admissions.empty()) {
        cout << "No admission records.\n";
        return;
    }

    cout << left << setw(8) << "AdmID"
        << setw(10) << "Patient"
        << setw(8) << "Bed"
        << setw(8) << "Days"
        << setw(10) << "Units"
        << setw(12) << "Status"
        << setw(12) << "Bill" << '\n';
    cout << string(68, '-') << '\n';

    for (const auto& a : admissions)
        cout << a << '\n';
}

void compareAdmissions() {
    int a1, a2;
    cout << "Enter first admission ID: ";
    cin >> a1;
    cout << "Enter second admission ID: ";
    cin >> a2;

    Admission* first = nullptr;
    Admission* second = nullptr;

    for (auto& a : admissions) {
        if (a.admissionId == a1) first = &a;
        if (a.admissionId == a2) second = &a;
    }
}

```

```

    if (!first || !second) {
        cout << "Invalid admission ID.\n";
        return;
    }

    if (*first > *second)
        cout << "Admission " << a1 << " has the higher bill.\n";
    else if (*second > *first)
        cout << "Admission " << a2 << " has the higher bill.\n";
    else
        cout << "Both admissions have equal bills.\n";
}

void utilizationReport() const {
    int occupied = 0;
    double revenue = 0.0;

    for (const auto& b : beds)
        if (b.occupied) ++occupied;

    for (const auto& a : admissions)
        if (!a.active) revenue += a.finalBill;

    int totalBeds = static_cast<int>(beds.size());
    int available = totalBeds - occupied;
    double utilization = totalBeds == 0 ? 0.0
        : (occupied * 100.0 / totalBeds);

    int active = 0;
    for (const auto& a : admissions)
        if (a.active) ++active;

    cout << "\n===== BED UTILIZATION REPORT =====\n";
    cout << "Total Beds      : " << totalBeds << '\n';
    cout << "Occupied Beds    : " << occupied << '\n';
    cout << "Available Beds   : " << available << '\n';
    cout << fixed << setprecision(2);
    cout << "Utilization      : " << utilization << "%\n";
    cout << "Active Admissions: " << active << '\n';
    cout << "Revenue Collected: Rs. " << revenue << '\n';
    cout << "Total Admissions : " << totalAdmissions << '\n';
}

void menu() {
    int choice;

    do {
        cout << "\n===== SMART HOSPITAL =====\n";
        cout << "1. Register Patient\n";
        cout << "2. Display All Patients\n";
        cout << "3. Display All Beds\n";
        cout << "4. Admit Patient / Allocate Bed\n";
        cout << "5. Update Monitoring or Service Usage\n";
        cout << "6. Calculate Current Bill\n";
        cout << "7. Discharge Patient\n";
        cout << "8. Display Admission Records\n";
        cout << "9. Compare Two Admissions\n";
        cout << "10. Generate Bed Utilization Report\n";
        cout << "0. Exit\n";
        cout << "Enter choice: ";
        cin >> choice;
    }
}

```

```

        switch (choice) {
            case 1: registerPatient(); break;
            case 2: displayPatients(); break;
            case 3: displayBeds(); break;
            case 4: admitPatient(); break;
            case 5: updateMonitoring(); break;
            case 6: calculateCurrentBill(); break;
            case 7: dischargePatient(); break;
            case 8: displayAdmissions(); break;
            case 9: compareAdmissions(); break;
            case 10: utilizationReport(); break;
            case 0: cout << "Exiting program...\n"; break;
            default: cout << "Invalid menu choice.\n";
        }
    } while (choice != 0);
}

};

int Hospital::totalAdmissions = 0;

int main() {
    Hospital hospital;
    hospital.menu();
    return 0;
}

```

4. Deliverable 3 – Sample Input / Output

```

SMART HOSPITAL MANAGEMENT SYSTEM
=====
1. Register New Patient
2. Discharge Patient
3. View All Patients
4. View All Beds
5. Billing Report
6. Patient Priority List
7. Exit
Enter your choice: 1
Enter Patient ID: 101
Enter Patient Name: Ramesh
Enter Patient Age: 45
Enter Diagnosis: Pneumonia
Select Patient Category:
1. General
2. ICU
3. Emergency
Enter choice: 2
Patient registered successfully and bed allocated!
Press Enter to continue...

Enter your choice: 1
Enter Patient ID: 102
Enter Patient Name: Suresh
Enter Patient Age: 30
Enter Diagnosis: Accident
Select Patient Category:
1. General
2. ICU
3. Emergency
Enter choice: 3
Patient registered successfully and bed allocated!
Press Enter to continue...

Enter your choice: 3
-----ALL-REGISTERED-PATIENTS-----
ID      Name      Age      Category      Diagnosis      Bed No      Daily Charge      Service Charge      Total Amount
-----
101     Ramesh      45       ICU           Pneumonia      1           7500.00           1000.00           8500.00
102     Suresh      30       Emergency     Accident       2           5000.00           750.00           5750.00
Press Enter to continue...

Enter your choice: 4
-----HOSPITAL-BEDS STATUS-----
Bed No      Ward Type      Status      Patient ID      Patient Name
-----
1           ICU           Occupied      101           Ramesh
2           Emergency     Occupied      102           Suresh
3           General       Available     -             -
4           General       Available     -             -
5           General       Available     -             -
Press Enter to continue...

Enter your choice: 5
-----BILLING REPORT-----
ID      Name      Category      Bed No      Daily Charge      Service Charge      Total Amount
-----
101     Ramesh      ICU           1           7500.00           1000.00           8500.00
102     Suresh      Emergency     2           5000.00           750.00           5750.00
-----
GRAND TOTAL: 14250.00
Press Enter to continue...

Enter your choice: 6
-----PATIENT-PRIORITY-LIST-(High to Low)-
ID      Name      Category      Priority      Monitoring Level
-----
102     Suresh      Emergency     4           2
101     Ramesh      ICU           3           3
Press Enter to continue...
Enter your choice: 7
Thank you for using Smart Hospital Management System!
Goodbye!

```

5. Deliverable 4 – Inheritance and Polymorphism Explanation

Inheritance is used because GeneralPatient, ICUPatient, and EmergencyPatient are all types of Patient but have different requirements. The common data such as patient ID, name, age, and diagnosis is kept in the Patient base structure. The derived structures override virtual functions for category, daily charge, service charge, priority, and monitoring requirements.

Polymorphism allows the Hospital to store different patient types through base-class pointers. At run time, calls such as getDailyCharge(), getServiceCharge(), getPriority(), and requiredMonitoring() select the correct derived implementation. Therefore the same admission and billing logic can work with all patient categories without separate billing code for every type.

- Patient → GeneralPatient
- Patient → ICUPatient
- Patient → EmergencyPatient
- virtual functions → runtime selection of patient-specific behaviour
- base-class pointer (Patient*) → points to different derived patient objects

6. OOP Concepts Demonstrated

Requirement	Implementation	Purpose
Encapsulation	private/protected data with public functions	Protect object data
Inheritance	Patient → General/ICU/Emergency	Reuse common patient features
Polymorphism	virtual overridden functions	Runtime-specific behaviour
Constructors	default, parameterized, copy	Initialize objects safely
Destructor	Patient, Admission, Hospital	Lifecycle/cleanup handling
Static members	Patient::totalRegistered, Hospital::totalAdmissions	Shared counts
Collections	vector of patients, beds, admissions	Manage multiple objects
Operator overloading	Admission::operator> and operator<<	Comparison and report formatting
Control structures	if, loops, switch, validation	Menu and workflow control

7. Billing Rules Used

Patient Type	Daily Bed Charge	Service/Monitoring Unit	Priority
General Patient	Rs. 2,000/day	Rs. 300	1
ICU Patient	Rs. 7,500/day	Rs. 1,000	3
Emergency Patient	Rs. 5,000/day	Rs. 750	4

Bill formula: Total Bill = (Number of Days × Daily Bed Charge) + (Service/Monitoring Units × Service Charge).

8. Validation and Error Handling

- Duplicate patient IDs are rejected.
- Age is validated to a reasonable range.
- Invalid patient categories are rejected.
- Unknown patient IDs are rejected.
- A patient cannot be admitted twice while an active admission exists.

- Unavailable or unsuitable beds produce a clear message.
- Monitoring/service units and stay days must be positive.
- Invalid menu choices are handled without terminating the program.

9. Deliverable 5 – Reflection and SDG Mapping

Design decision: The structure-based organization keeps each entity's data and operations together. The Patient hierarchy is appropriate because all patient categories share common information but differ in charges, monitoring needs, and priority.

Constructors initialize patient, bed, and admission objects. Copy constructors demonstrate safe object copying. Destructors provide lifecycle cleanup points; the Hospital destructor releases any still-occupied beds before the hospital object is destroyed.

Operator overloading is meaningful because `operator>` provides a natural way to compare two admission records by bill, while `operator<<` provides a clean way to print an admission record.

Implementation challenge: managing the relationship between patients, beds, and admissions can cause inconsistent states. This implementation solves it by using patient IDs and bed numbers as associations, checking active admissions before allocation, and releasing the corresponding bed during discharge.

SDG 3 – Good Health and Well-being: The system supports organized patient admission, bed allocation, monitoring/service tracking, and discharge billing, which can improve hospital workflow and resource visibility.

SDG 9 – Industry, Innovation and Infrastructure: The project demonstrates a software-based automation approach for hospital infrastructure management, replacing manual tracking with a structured digital workflow.

10. Deliverable 6 – GitHub Upload

Github Repository Link : <https://github.com/dharshidha07-bad/smart-hospital-bed-patient-monitoring-management-system-using-c/blob/main/README.md>

11. Quick Viva / Presentation Keywords

- Encapsulation – protect data using access specifiers.
- Inheritance – derive specialized patient categories from Patient.
- Polymorphism – virtual functions select behaviour at runtime.
- Constructor – initializes objects.
- Destructor – handles object lifecycle cleanup.
- Static – shared hospital-level counters.
- Vector – collection of objects/pointers.
- `operator>` – compares admission bills.
- `operator<<` – formats admission output.
- Bed allocation – selects a suitable free bed.
- Billing – daily charge + service usage.
- Utilization – $\text{occupied beds} / \text{total beds} \times 100$.

12. Requirement-to-Code Mapping

Assignment Requirement	Where Demonstrated
Patient registration	Hospital::registerPatient
Bed management	HospitalBed + Hospital::displayBeds
Bed allocation	Hospital::allocateBed / admitPatient
Admission management	Admission + Hospital::admitPatient
Monitoring update	Hospital::updateMonitoring
Billing	Admission::calculateBill
Discharge	Hospital::dischargePatient
Admission comparison	Admission::operator>
Utilization report	Hospital::utilizationReport
Menu interface	Hospital::menu
Inheritance/polymorphism	Patient hierarchy and virtual functions
Constructors/destructors	Patient, Admission, Hospital
Static members	Patient::totalRegistered, Hospital::totalAdmissions
Operator overloading	operator> and operator<<

13. Note

The assignment source specifies the functional behaviour, required OOP concepts, suggested class structure, billing values, menu, reflection, SDG mapping, and deliverables. This document follows those requirements and uses structures for the C++ implementation as requested.